体系结构风格与设计过程

软件工程与计算 II · 体系结构模块 · 第 2 讲

南京大学 · 刘钦

今日学习路径



今日核心问题

1. 常见体系结构风格有哪些？各自**适用场景**是什么？
2. 分层风格和 MVC 风格如何**指导**系统设计？
3. 从需求到架构的**设计过程**包含哪些步骤？

预习测试

1. 体系结构设计的过程包含哪些步骤？请简要描述。
2. 列举一个**非功能性需求**对体系结构设计的影响，并说明原因。

预习测试 — 参考思路

设计过程步骤

1. 分析关键需求与约束
2. 选择体系结构风格
3. 进行逻辑设计（子系统划分）
4. 设计接口与数据流
5. 评审与迭代

非功能需求影响示例

高并发 → 采用事件驱动或微服务风格，而非单体分层
安全性 → 增加认证/授权层，影响分层数量
可移植性 → 严格分层隔离平台依赖

PART 1 · 体系结构风格概览

Architectural Styles Overview

体系结构风格的概念

体系结构风格（Architectural Style）是一组**设计决策的集合**，它规定了：

- 一组**组件类型**（如过滤器、层、服务）
- 组件之间的**连接方式**（如管道、调用、事件）
- 组件交互的**约束规则**（如分层只能调用相邻层）

> 风格 = **组件词汇** + **连接器词汇** + **拓扑约束**

为什么需要风格?

- 提供**经验复用** — 前人验证过的设计方案
- 提供**通用词汇** — 团队沟通的共同语言
- 明确**质量属性**权衡 — 每种风格有特定的优劣

体系结构风格分类总览

风格	组件	连接器	典型应用
分层 (Layered)	层 (Layer)	过程调用	企业信息系统
管道–过滤器 (Pipe–Filter)	过滤器	管道（数据流）	编译器、数据处理
客户端–服务器 (C/S)	客户端、服务器	请求/响应	Web 应用、数据库
事件驱动 (Event–Driven)	事件源、监听器	事件总线	GUI、消息系统
微服务 (Microservices)	独立服务	REST/消息队列	大规模互联网系统
主程序–子程序	主程序、子程序	过程调用	传统结构化系统

风格 vs 模式 —— 区别 [SWEBOK 2.2]

维度	体系结构风格 (Style)	体系结构模式 (Pattern)
作用范围	系统**全局**组织方式	**局部**重复问题的解决方案
粒度	定义系统大部分的组织	可应用于系统中的单个元素
示例	分层、微服务、管道–过滤器	观察者、MVC、代理
能否共存	一个系统通常选择**一种**主风格	多个模式可以**同时存在**

案例：在线书店中风格与模式共存

- **风格**：三层分层架构（全局组织）
- **模式**：MVC（展示层内部）、Observer（模型通知视图）、DAO（数据访问层）

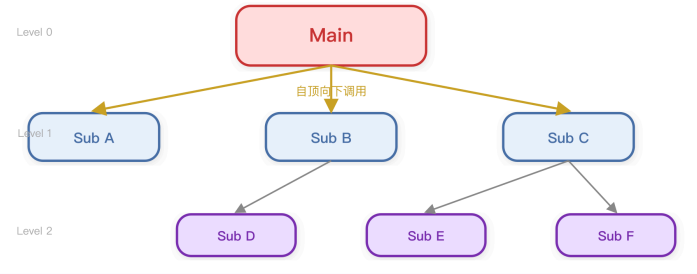
SWEBOK 体系结构风格六大类 [SWEBOK 2.2]

#	类别	包含的风格
1	General Structures	分层、管道–过滤器、黑板
2	Distributed Systems	客户端–服务器、n–tier、发布–订阅、REST、微服务
3	Method–Driven	面向对象、事件驱动、数据流
4	User–Computer Interaction	MVC、PAC（Presentation–Abstraction–Control）
5	Adaptive Systems	微内核、反射、元级架构
6	Virtual Machines	解释器、规则引擎、进程控制

在线书店适用：分层（#1）+ REST（#2）+ 面向对象（#3）+ MVC（#4）

主程序–子程序风格

结构特征



- 自顶向下的**调用层次**
- 数据通过参数传递

优缺点

优势	劣势
结构清晰	耦合度高
易于理解	难以复用
控制流明确	不支持并发
调试方便	扩展性差

适合**小规模、逻辑简单**的系统

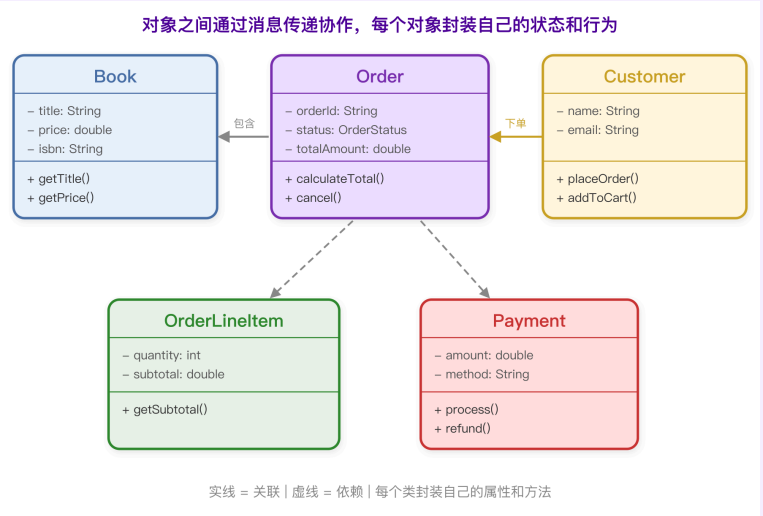
面向对象风格

面向对象风格以**对象**为基本组件，通过**方法调用**进行交互。

三大特性对架构的影响

特性	架构影响
封装	信息隐藏，降低耦合
继承	代码复用，建立层次
多态	灵活替换，开闭原则

案例：在线书店对象协作



OO 风格是现代分层架构和 MVC 的**基础**——对象封装数据与行为，通过消息传递协作

PART 2 · 分层风格

Layered Architecture Style

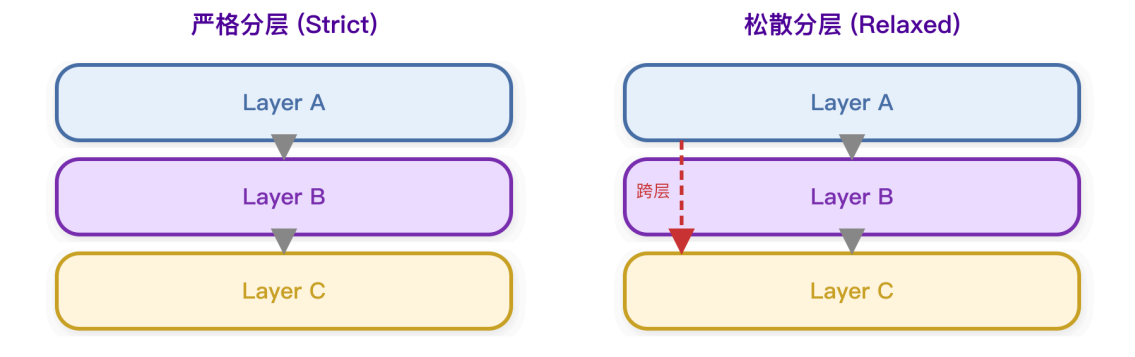
分层体系结构原理

严格分层 (Strict Layering)

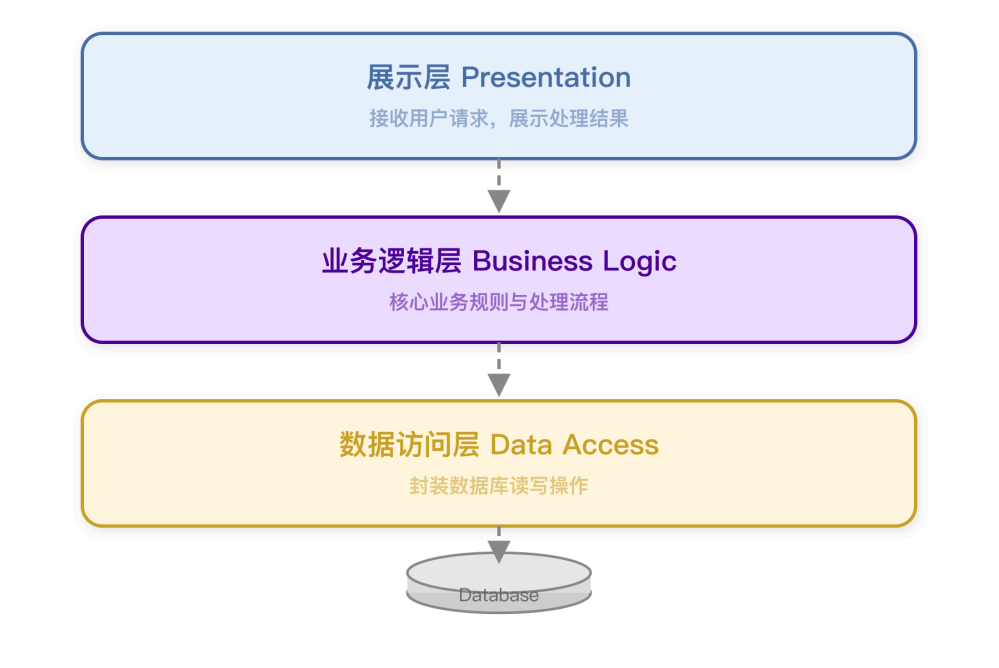
- 每层**只能调用直接下层**
- 层间依赖关系清晰
- 修改影响范围可控
- 例：OSI 七层网络模型

松散分层 (Relaxed Layering)

- 允许**跨层调用**
- 提高运行效率
- 降低开发复杂度
- 例：大多数企业应用



三层架构详解



核心原则：上层依赖下层，下层不依赖上层；每层通过**接口**暴露服务。

三层架构 — 各层职责

层次	英文名	核心职责	典型技术
展示层	Presentation	接收请求、渲染视图、用户交互	JSP, Thymeleaf, Vue, React
业务逻辑层	Business Logic	实现业务规则、协调工作流	Spring Service, EJB
数据访问层	Data Access	CRUD 操作、数据映射、事务管理	MyBatis, JPA, Hibernate

关键设计约束

- 展示层**不直接访问**数据库
- 业务逻辑层**不包含** UI 代码
- 数据访问层**不包含**业务规则
- 各层通过**接口（Interface）**解耦

分层风格的四大优势

优势	说明	示例
封装性	每层内部实现对外隐藏	切换 MySQL 到 PostgreSQL 不影响业务层
关注点分离	开发者只需关注本层逻辑	前端工程师无需了解 SQL
可替换性	整层可被替换而不影响其他层	将 Web UI 替换为移动端 UI
可测试性	各层可独立单元测试	Mock 数据层测试业务逻辑

分层的代价

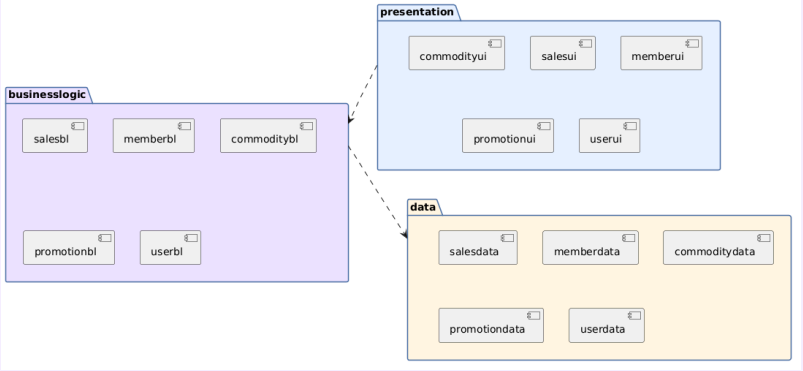
- **性能开销** — 层间调用带来额外开销
- **级联修改** — 新增字段可能需要逐层传递
- **过度分层** — 简单系统分层反而增加复杂度

案例：连锁超市系统

基本功能模块

1. **销售** — 收银、退货、结算
2. **库存管理** — 入库、出库、盘点
3. **商品管理** — 商品信息维护
4. **会员管理** — 注册、积分、等级
5. **促销策略** — 满减、折扣、赠品
6. **财务管理** — 收支记录、报表
7. **人员管理** — 员工信息、权限
8. **日志审计** — 操作记录、异常

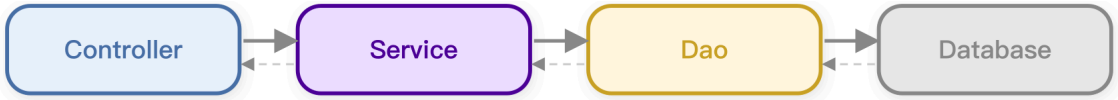
三层包图



案例：在线书店系统分层架构

层次	包名	关键类
展示层	`presentation`	`BookController`, `OrderController`, `UserController`
业务逻辑层	`service`	`BookService`, `OrderService`, `UserService`
数据访问层	`dao`	`BookDao`, `OrderDao`, `UserDao`
实体层	`entity`	`Book`, `Order`, `User`, `CartItem`

调用链路



在线书店 — 分层代码示例

```
// === 展示层 Controller ===
@RestController
public class BookController {
    @Autowired private BookService bookService;

    @GetMapping("/books/{id}")
    public Book getBook(@PathVariable Long id) {
        return bookService.findById(id);  // 调用业务层
    }
}

// === 业务逻辑层 Service ===
@Service
public class BookService {
    @Autowired private BookDao bookDao;

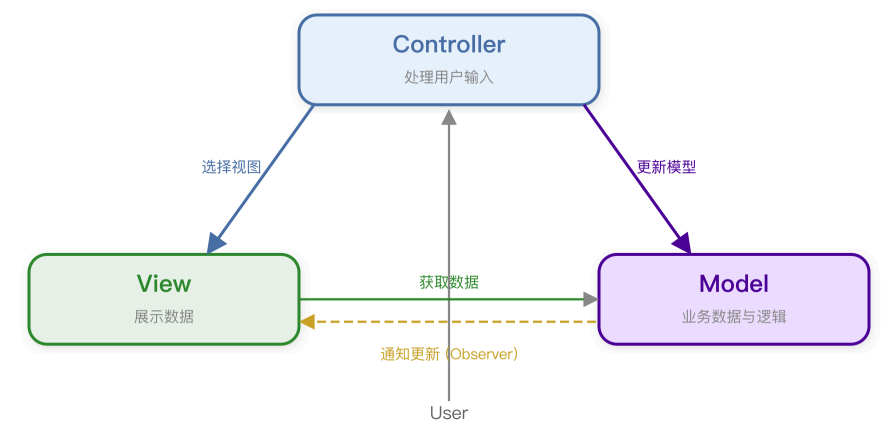
    public Book findById(Long id) {
        return bookDao.selectById(id);    // 调用数据层
    }
}

// === 数据访问层 Dao ===
@Mapper
public interface BookDao {
    @Select("SELECT * FROM book WHERE id = #{id}")
    Book selectById(Long id);             // 访问数据库
}
```

PART 3 · MVC 风格

Model–View–Controller Pattern

MVC 三组件职责



组件	职责	要点
Model	管理数据与业务逻辑	不依赖 View/Controller
View	展示数据给用户	从 Model 获取数据
Controller	处理用户输入	更新 Model，选择 View

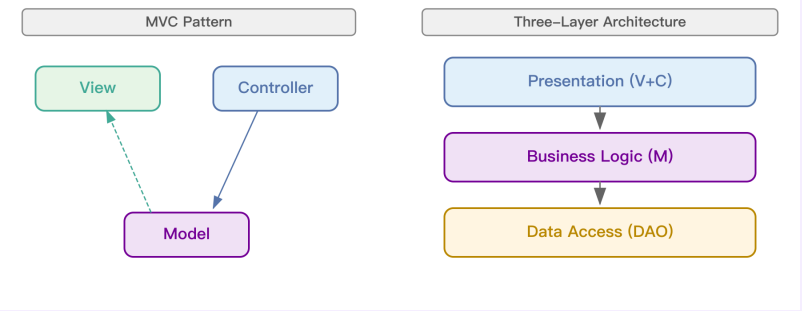
核心原则

- Model **独立于**表现形式
- View 只负责**展示**，不含业务逻辑
- Controller 是 View 与 Model 的**协调者**

MVC 与分层风格的关系

映射关系

MVC	三层架构
View	展示层
Controller	展示层
Model	业务逻辑层 + 数据访问层



实践中常将 MVC **嵌入**展示层

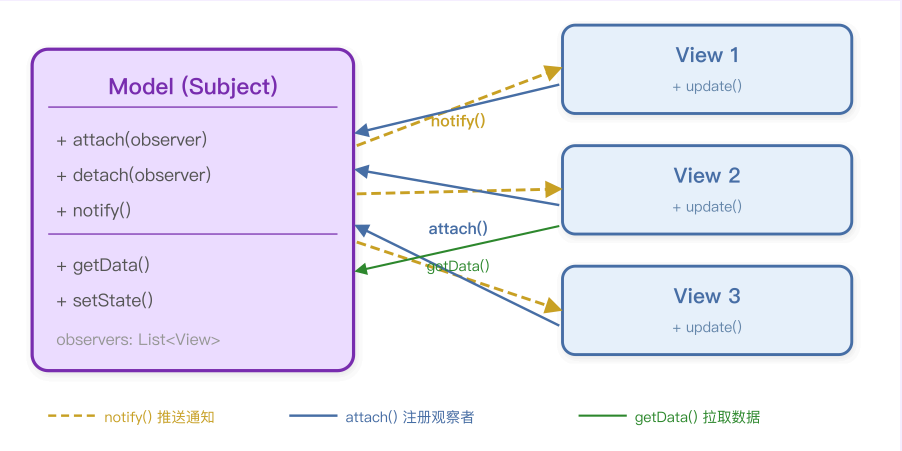
关键区别

- **分层**关注**纵向**职责划分
- **MVC**关注**交互**逻辑分离
- 两者可以**结合使用**

观察者模式在 MVC 中的角色

观察者模式 (Observer Pattern)

- Model 是**被观察者** (Subject)
- View 是**观察者** (Observer)
- Model 数据变化时**通知**所有注册的 View
- View 收到通知后**自动刷新**



好处

- Model 与 View **松耦合**
- 支持**多视图**同步更新
- 新增 View 无需修改 Model

案例：在线书店 MVC 代码

```
// === Controller 层 ===
@Controller
public class BookController {
    @Autowired private BookService bookService;

    @GetMapping("/books")
    public String listBooks(Model model) {
        List<Book> books = bookService.findAll();
        model.addAttribute("books", books);   // 传递数据给 View
        return "bookList";                    // 选择 View 模板
    }

    @PostMapping("/books")
    public String addBook(@RequestBody Book book) {
        bookService.save(book);               // 更新 Model
        return "redirect:/books";             // 重定向刷新
    }
}

// === Service 层（Model 的一部分）===
@Service
public class BookService {
    @Autowired private BookDao bookDao;

    public List<Book> findAll() { return bookDao.selectAll(); }
    public void save(Book book) { bookDao.insert(book); }
}
```

管道–过滤器风格（Pipe–and–Filter）[SWEBOK 2.2]



案例：Java Stream API

```
List<String> titles = books.stream()
    .filter(b -> b.getPrice() > 50)
    .sorted(comparing(Book::getPrice))
    .map(Book::getTitle)
    .collect(toList());
```

案例：Unix 管道

```
cat access.log | grep "POST /api/orders"
  | awk '{print $1}' | sort | uniq -c
```

客户端–服务器 + REST 风格 [SWEBOK 2.2]

REST 核心约束 [Fielding 2000]

- **无状态**：每次请求包含所有信息
- **统一接口**：HTTP 动词 + URI 资源
- **分层系统**：客户端不知道是否直连服务器

在线书店 RESTful API

方法	端点	功能
GET	`/api/books`	搜索图书
GET	`/api/books/{id}`	图书详情
POST	`/api/orders`	创建订单
PUT	`/api/cart/items`	更新购物车
DELETE	`/api/cart/items/{id}`	删除商品

Spring Boot 实现

```
@RestController
@RequestMapping("/api/books")
public class BookController {

    @Autowired
    private BookService bookService;

    @GetMapping("/{id}")
    public Book getBook(
            @PathVariable String id) {
        return bookService.findById(id);
    }

    @GetMapping
    public Page<Book> search(
            @RequestParam String keyword,
            @RequestParam int page) {
        return bookService.search(
            keyword, page);
    }
}
```

微服务风格 [SWEBOK 2.2]

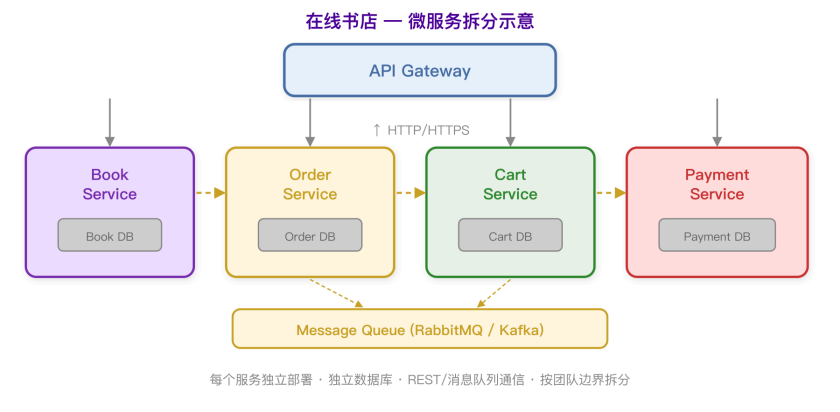
核心特征

- 每个服务**独立部署**、独立数据库
- 通过 REST/消息队列通信
- 团队按服务边界划分（Conway 定律）

单体 vs 微服务对比

维度	单体	微服务
部署	整体部署	独立部署
扩展	整体扩展	按需扩展
技术栈	统一	可异构
复杂度	代码内	运维层
适合	小团队/早期	大团队/高流量

在线书店微服务拆分示意

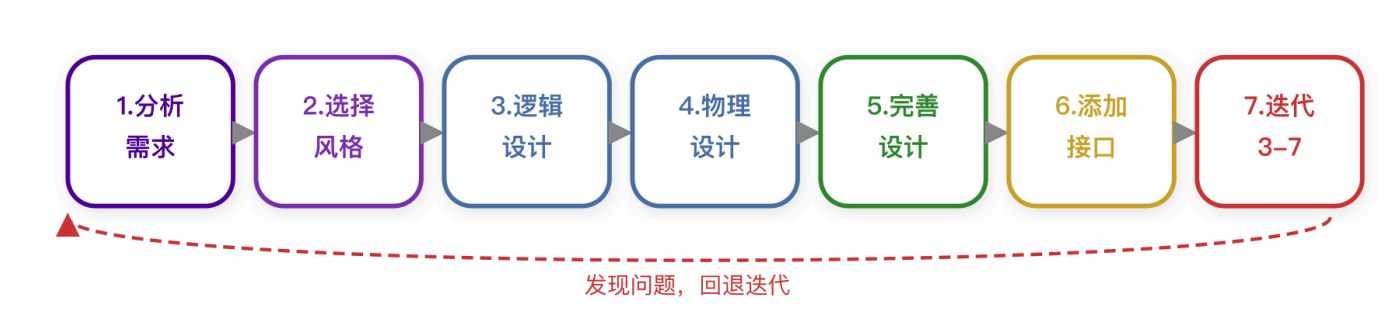


5 人团队 + 无 K8s 经验 → 选择**单体分层**更合适

PART 4 · 体系结构设计过程

Architectural Design Process

体系结构设计过程 — 简化版



设计过程是**迭代**的，在任何步骤都可以回退到前面的步骤进行调整。

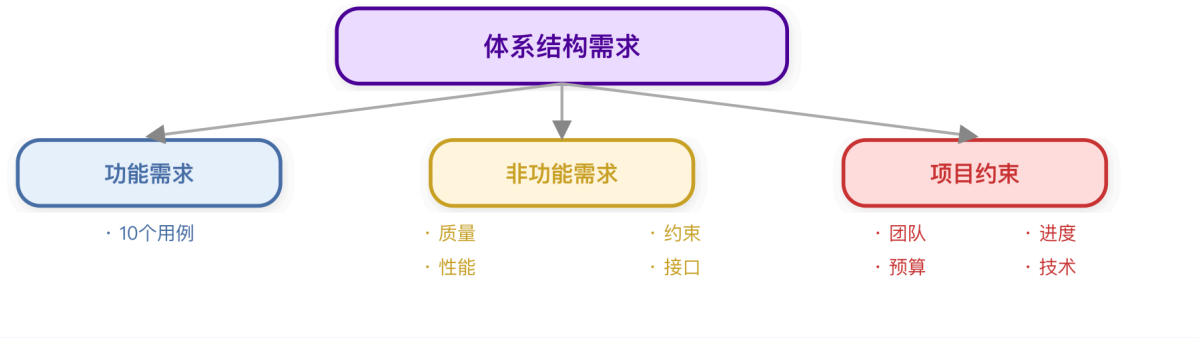
体系结构设计过程 — 完整版

步骤	名称	核心任务
1	分析关键需求	识别功能需求、质量属性、项目约束
2	选择体系结构风格	根据需求特征选择合适风格
3	逻辑设计	将需求分配到子系统和模块
4	接口设计	定义模块间的交互接口
5	物理设计	决定部署方案（硬件/网络拓扑）
6	验证设计方案	检查是否满足需求和约束
7	评审与改进	团队评审，迭代优化

关键提示：步骤 3（逻辑设计）是最核心的步骤，需要反复迭代直到模块划分合理。

步骤 1：分析关键需求和项目约束

体系结构需求的构成



需求分类详述

非功能需求

类型	示例
质量属性	可维护性、可扩展性、安全性
性能约束	响应时间 < 2s，并发 > 1000
接口需求	兼容第三方支付、对接ERP

项目约束

类型	示例
团队	5 人团队，2 名初级开发
预算	总预算 50 万
进度	3 个月交付
技术	必须使用 Java + Spring

非功能需求和项目约束往往对架构选择的影响**大于**功能需求。

ASR —— 架构重要需求 [SWEBOK 3.2.1]

Architecturally Significant Requirement (ASR)：影响软件体系结构的需求。
并非所有需求都是 ASR——只有那些**驱动架构决策**的需求才是。

案例：在线书店 ASR 识别

需求	是否 ASR	理由
支持 1000 并发用户	**ASR**	驱动缓存/负载均衡决策
支付信息加密传输	**ASR**	驱动安全层/HTTPS 决策
按书名搜索图书	非 ASR	功能需求，不影响整体架构
按钮颜色为蓝色	非 ASR	UI 细节，与架构无关
系统需用 Java 开发	**ASR**	技术约束，影响全部技术选型
7x24 可用，年可用率 99.9%	**ASR**	驱动冗余部署/故障转移决策

架构设计三阶段循环 [SWEBOK 3.2] [Hofmeister 2007]

Analysis → Synthesis → Evaluation（Figure 2.3）

阶段	活动	产出
Analysis	收集 ASR，理解利益相关人关注点	ASR 列表、系统约束
Synthesis	构建候选架构方案，做权衡	候选架构、设计决策
Evaluation	验证方案是否满足 ASR	评审结果、改进建议

三个阶段**迭代进行**：评估发现问题 → 回到分析重新理解 → 再次综合新方案

这是所有架构设计方法（ADD/ATAM/QAW）的**底层循环模型**

步骤 2：选择体系结构风格

需求特征	推荐风格	原因
企业级业务系统、团队分工	**分层**	职责分明、易于分工
数据流式处理、转换链	**管道–过滤器**	数据逐步变换
高交互、多视图同步	**MVC**	界面与逻辑分离
大规模、独立部署、多团队	**微服务**	独立开发/部署/扩展
松耦合、异步通信	**事件驱动**	发布/订阅解耦
小型工具、脚本	**主程序–子程序**	简单直接

实践建议：大多数企业系统首选**分层架构**，在此基础上根据需要引入其他风格。

步骤 3：逻辑设计 — 需求到模块的映射

核心任务：将功能需求分配到各子系统和模块中。

设计原则

1. **高内聚** — 相关功能放在同一模块
2. **低耦合** — 模块间依赖最小化
3. **单一职责** — 每个模块只做一件事
4. **信息隐藏** — 隐藏实现细节，暴露接口

步骤

1. 识别**功能域**（Sales, Inventory, Member ...）
2. 为每个功能域创建**三层模块**（UI, BL, Data）
3. 定义模块间的**依赖关系**
4. 检查模块**粒度**是否合理

案例：连锁超市 — 功能到逻辑包映射

功能域	展示层（UI）	业务逻辑层（BL）	数据层（Data）
销售	SalesUI	Sales	SalesData
库存	InventoryUI	Inventory	InventoryData
商品	CommodityUI	Commodity	CommodityData
会员	MemberUI	Member	MemberData
促销	PromotionUI	Promotion	PromotionData
财务	FinanceUI	Finance	FinanceData
人员	StaffUI	Staff	StaffData

每个功能域在三层中各有**对应的包**，形成**纵向切片**。

改进后的概要功能设计

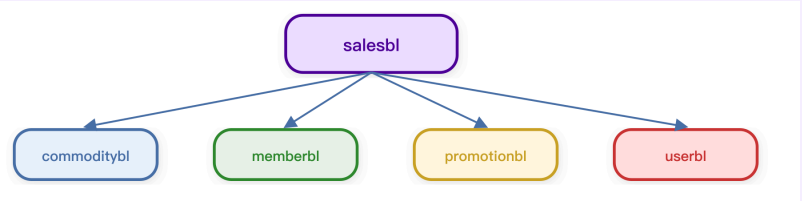
问题：跨模块数据依赖

销售功能不仅需要 `Sales` 自身的数据，还需要：

- **Commodity** — 商品信息查询
- **Member** — 会员折扣计算
- **Promotion** — 促销策略应用
- **Finance** — 收款记账

解决方案

模块间通过**接口调用**而非直接数据访问。



> 业务逻辑层内部的**横向调用**是正常的，关键是通过接口解耦。

从逻辑设计到评审：迭代过程



常见迭代场景

场景	回退到	原因
接口设计时发现模块职责不清	步骤 3	重新划分模块
物理部署时发现性能瓶颈	步骤 2/3	调整风格或拆分模块
验证时发现安全需求遗漏	步骤 1	补充需求分析
评审时团队对设计方案有分歧	步骤 3	重新讨论模块划分

步骤 4–7 概述

步骤 4：接口设计

- 定义模块对外暴露的 **API**
- 确定参数、返回值、异常
- 使用 Interface / DTO 解耦

步骤 5：物理设计

- 决定**部署拓扑**（单机/分布式）
- 数据库选型与部署
- 网络、中间件配置

步骤 6：验证设计方案

- 对照需求**逐条检查**
- 场景走查 (Scenario Walkthrough)
- 原型验证关键风险点

步骤 7：评审与改进

- 组织**架构评审会**
- 检查一致性与完整性
- 记录**架构决策记录** (ADR)
- 输出《架构设计文档》

PART 5 · 案例：从需求到架构

From Requirements to Architecture

连锁超市实践案例 — 需求分析

功能需求（10 个）

1. 商品信息管理
2. 销售收银处理
3. 退货管理
4. 库存盘点
5. 会员注册与管理
6. 促销策略管理
7. 财务收支记录
8. 员工信息管理
9. 日志审计记录
10. 经营报表生成

非功能需求与约束

编号	类型	描述
Security1	安全	用户身份认证
Security2	安全	权限分级控制
Security3	安全	操作日志审计
IC2	约束	需兼容已有 ERP

项目约束

- **技术栈**：Java + Spring Boot
- **进度**：开发周期紧张
- **团队**：5 人开发团队

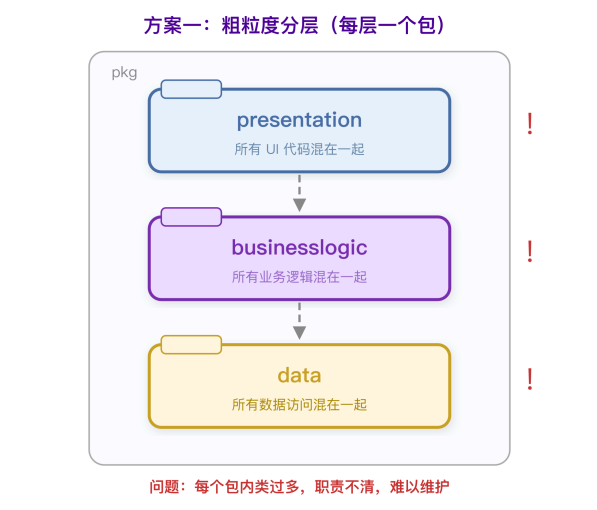
选择分层风格的理由

考虑因素	分析	结论
系统类型	企业级业务信息系统	适合分层
团队技术栈	Java + Spring（天然分层）	适合分层
非功能需求	安全（认证/鉴权可作为独立层）	适合分层
可维护性	功能模块多，需要清晰职责划分	适合分层
项目约束	进度紧，分层架构成熟度高	适合分层
兼容约束	IC2 需在数据层封装 ERP 接口	适合分层

决策：采用**三层分层架构** + 松散分层策略

初步设计方案一 — 问题分析

方案一结构（每层只有一个包）



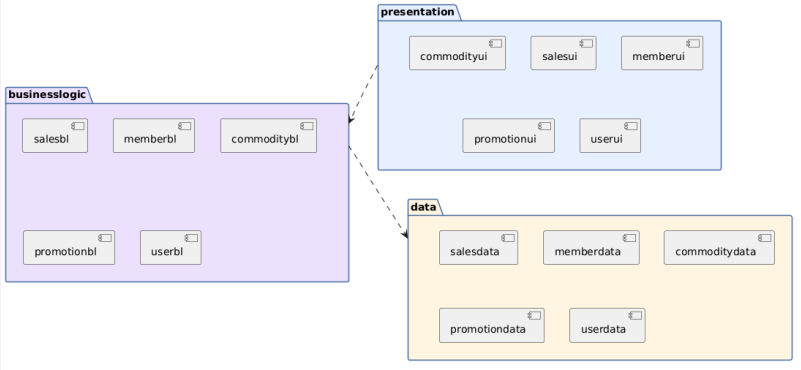
问题：数据层粒度太粗

Database 只有一个包，所有数据访问逻辑混在一起：

- 销售数据 + 库存数据 + 会员数据 ...
- 违反**单一职责原则**
- 无法独立修改某功能的数据访问
- 不同功能的 SQL 耦合在一起
- 团队成员并行开发困难

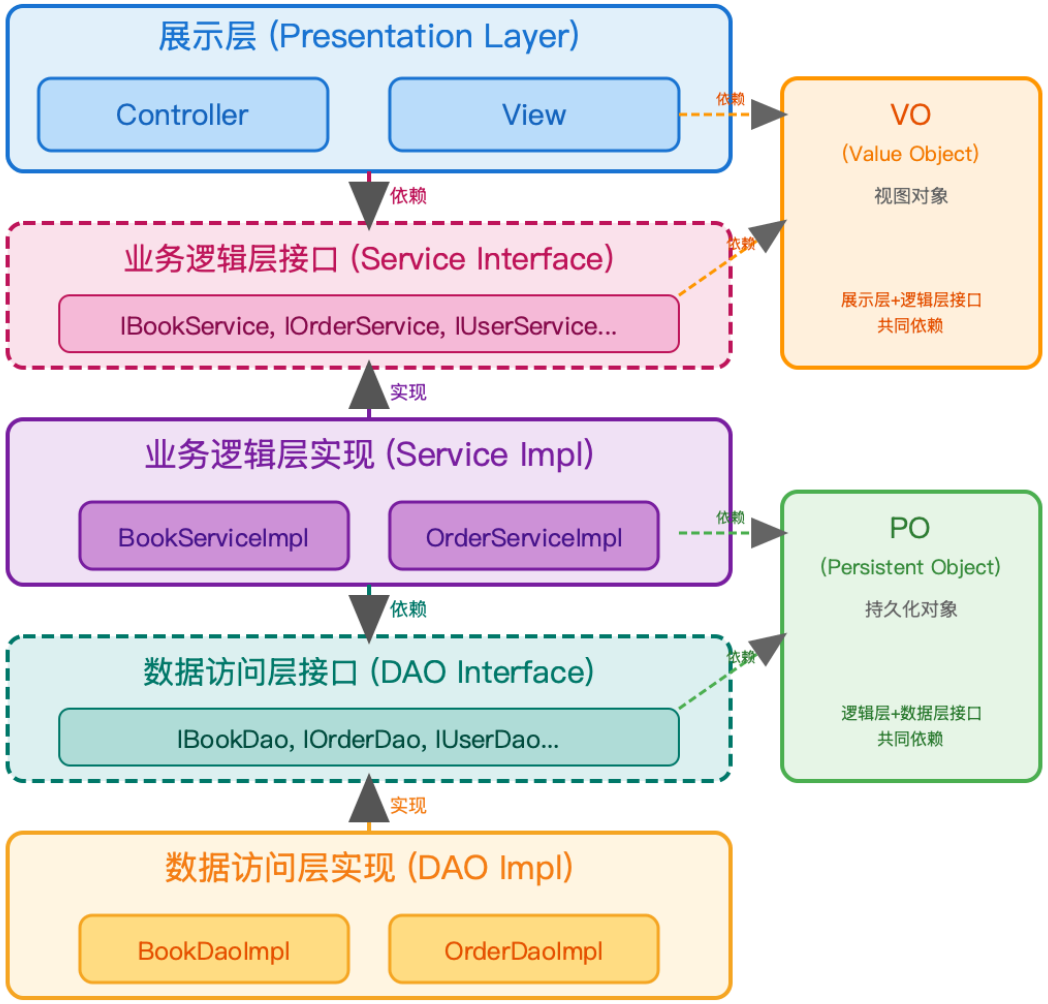
数据层需要按功能域**细分**为多个子包。

初步设计方案二 — 改进版



改进：三层均按功能域拆分为独立子包，每层约 5 个包，粒度合理，支持团队并行开发。

初步设计方案三 — 接口独立层与依赖倒置



核心改进

改进点	说明
接口独立成层	Service Interface 和 DAO Interface 作为独立层
实现继承接口	Service Impl → implements → Service Interface
上层依赖接口	展示层依赖 Service Interface，不依赖实现
VO/PO分离	VO用于展示层–逻辑层，PO用于逻辑层–数据层

依赖关系

依赖方向	说明
展示层 → Service Interface	上层依赖接口
Service Impl → Service Interface	实现继承接口
Service Impl → DAO Interface	上层依赖接口
DAO Impl → DAO Interface	实现继承接口
展示层 + Service Interface → VO	共同依赖值对象
Service Impl + DAO Interface → PO	共同依赖持久化对象

PART 6 · AI 增强：架构风格选择

AI-Augmented Architectural Decision

LLM × 软件架构：研究前沿 [arxiv 2505.16697, 2025]

系统综述：18 篇文献的关键发现

发现	说明
LLM 主要应用于**初期阶段**	Requirements-to-Architecture、Architecture-to-Code
OpenAI GPT 模型**主导**	少数使用 Claude、Gemini、开源模型
Few-shot + RAG 是主要技术	Prompt 工程是关键成功因素
LLM 通常**优于基线**	但在复杂架构推理中仍有局限

仍待探索的领域

- 架构**一致性检查**（Conformance Checking）
- **云原生**架构自动生成
- 架构**演化**与重构建议
- 多视图**一致性验证**

工业实践：Spec 驱动架构生成 [2025]

工具	公司	架构相关能力
AWS Kiro	Amazon	Spec → Design → Tasks → Code 全流水线；Spec 是"超级 Prompt"
GitHub Spec Kit	GitHub	开源 SDD 工具包，支持 22+ AI Agent 平台
Claude Code	Anthropic	CLAUDE.md 作为项目"宪法"；Subagent 并行架构任务
Devin / Codex	Cognition / OpenAI	全栈 Agent，隐式做出架构决策

挑战

Agent 的架构决策**速度快但不透明**。
推荐做法：让 Agent 生成架构方案 → **人工 Review** → 记录 ADR → 再让 Agent 实现。

LLM 辅助架构风格选择

Prompt 示例

```
你是一位资深软件架构师。请根据以下需求特征，推荐最合适的
体系结构风格并给出理由：

【系统类型】连锁超市管理系统
【功能需求】销售收银、库存管理、会员管理等 10 个模块
【非功能需求】安全性高、需兼容已有 ERP
【团队规模】5 人，Java 技术栈
【项目约束】3 个月交付

请从以下风格中选择：分层、微服务、事件驱动、管道–过滤器
对每种风格给出适用性评分（1–5）和关键理由。
```

使用 AI 的目的：快速获得多角度分析，而非直接采纳结论。

AI 对比分析：分层 vs 微服务

质量属性	分层架构	微服务架构	本项目倾向
可维护性	中高（层内修改隔离）	高（服务独立）	分层够用
可扩展性	中（垂直扩展为主）	高（水平扩展）	分层够用
开发效率	高（成熟框架支持）	低（基础设施复杂）	**分层优**
部署复杂度	低（单体部署）	高（容器编排）	**分层优**
团队要求	低（通用技能）	高（DevOps 技能）	**分层优**
性能	中高（进程内调用）	中（网络通信开销）	**分层优**

AI 结论：5 人团队、3 个月周期、Java 技术栈 → 分层架构是最优选择，微服务引入的复杂度远超收益。

人工审查 AI 架构建议的检查清单

检查项	具体问题
需求覆盖	AI 建议是否覆盖了所有关键需求?
约束一致	建议是否与项目约束（预算/进度/团队）一致?
经验验证	团队是否有该风格的实际经验?
风险评估	AI 是否识别了该方案的主要风险?
替代方案	是否考虑了足够的备选方案?
演化能力	该方案是否支持系统未来的演化?
技术生态	推荐的技术栈是否有成熟的社区和工具支持?

切记：AI 生成的架构建议必须经过人工审查。AI 可能缺乏项目上下文，给出看似合理但不适合具体场景的建议。

思政：技术决策的长远影响与可持续性

架构决策的社会责任

- **可维护性** = 对后续开发者的尊重
- **安全性** = 对用户数据的保护
- **可扩展性** = 对业务发展的预见
- **文档化** = 对知识传承的负责

技术债务的代价

> 今天的捷径 = 明天的负担

短期利益驱动的架构决策可能导致系统在数年后变得难以维护，最终损害用户利益和团队士气。

可持续软件工程

原则	实践
绿色设计	减少不必要的资源消耗
长期视角	架构决策考虑 3–5 年演化
知识共享	ADR 记录决策上下文
技术伦理	数据隐私、算法公平

> 优秀的架构师不仅关注技术指标，更关注技术决策对**人**和**社会**的影响。

PART 7 · 总结与作业

Summary and Assignment

课堂总结

主题	要点
体系结构风格	风格 = 组件词汇 + 连接器 + 拓扑约束
分层风格	三层架构：展示层/业务逻辑层/数据访问层
MVC 风格	Model–View–Controller 分离交互逻辑
设计过程	7 步迭代：需求→风格→逻辑→接口→物理→验证→评审
案例实践	连锁超市：需求分析→选择分层→逐步细化模块
AI 增强	LLM 辅助风格选择，但必须人工审查

核心收获

1. 选择架构风格要基于**需求特征**和**项目约束**，而非技术偏好
2. 逻辑设计的关键是**合理的模块粒度**和**清晰的职责划分**
3. 架构设计是**迭代过程**，不是一次性完成

课后作业

任务：学生选课系统 — 业务逻辑层设计

画出**选课业务逻辑** `chooseCoursebl` 的关键类图，要求包含：

1. **核心类**：`CourseService`，`StudentService`，`EnrollmentService`
2. **实体类**：`Course`，`Student`，`Enrollment`
3. 类之间的**关联关系**（依赖、关联、组合）
4. 关键方法签名（至少 3 个核心方法）

提交要求

- 使用 UML 类图标准符号
- 标注类的属性和方法
- 用文字简述设计理由（200 字以内）

> **提示**：先识别功能需求（选课、退课、查课表），再按职责分配到各个类中。

下节课预告

03–03 体系结构设计实践与验证

内容提要

1. 体系结构设计文档编写
2. 架构评估方法（ATAM）
3. 架构决策记录（ADR）
4. 从架构到详细设计的过渡
5. 实战：完成选课系统架构设计

预习任务

- 阅读教材第 9 章相关内容
- 回顾 ATAM 评估方法的基本流程
- 思考：如何评估一个架构方案的**可维护性**？

体系结构风格与设计过程

感谢聆听 · 欢迎提问

南京大学 · 刘钦